

Relatório

Aluno: João Vitor Garcia Aguiar Mintz

RA: 10440421

Link do vídeo: https://youtu.be/PkUTrX_dFdo

Para pensar: No meu código, caso dois usuários tentem enviar lances simultaneamente, ambos terão seus lances aceitos, ocorrendo, portanto, a race condition. Isso acontece quando duas ou mais threads acessam e modificam concorrentemente um recurso compartilhado e o resultado final depende da ordem em que essas operações são executadas, podendo fazer com que uma atualização sobrescreva a outra. Para evitar esse problema, é necessário controlar o acesso ao recurso compartilhado. Para isso, é comum utilizar mecanismos de sincronização, como Mutex ou Locks, que garantem que apenas uma thread por vez execute determinado trecho crítico do código. É importante ressaltar que o uso desses mecanismos pode afetar a eficiência do sistema, pois transforma uma parte que poderia ser executada em paralelo em uma seção sequencial, reduzindo o grau de paralelismo e podendo gerar perda de desempenho.